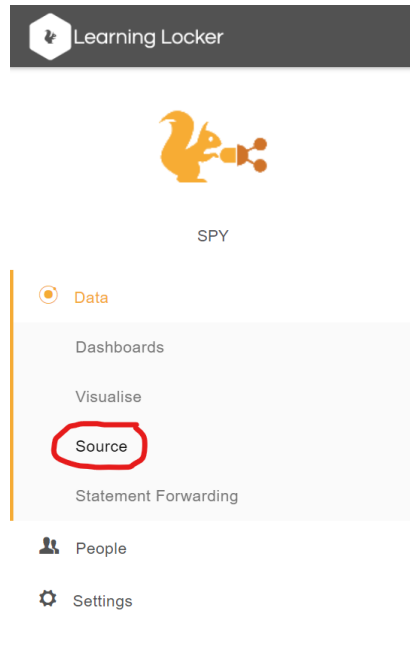


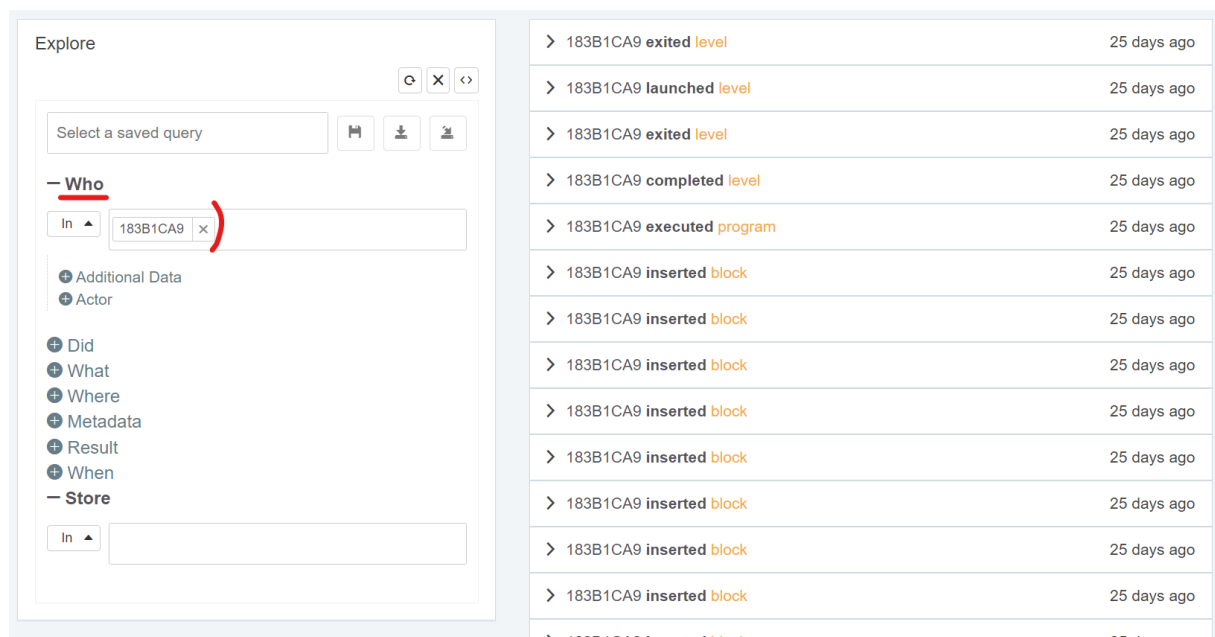
Tutoriel de récupération des données du LRS de SPY

Export des données depuis LearningLocker

1. Connectez-vous au LRS de SPY en suivant les instructions données sur la page suivante : <https://spy.lip6.fr/openTraces.html>
2. Accédez aux sources

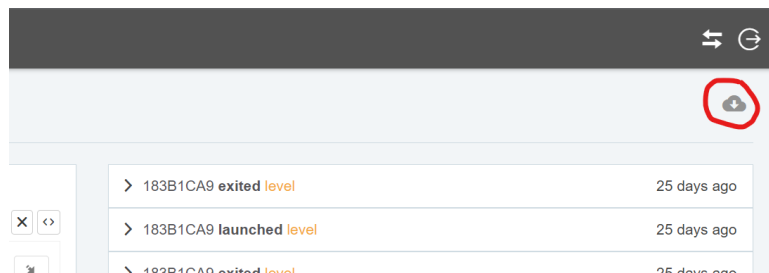


3. Filtrez les données en ne sélectionnant par exemple que votre session de jeu (remplacez le code de session par votre propre identifiant)

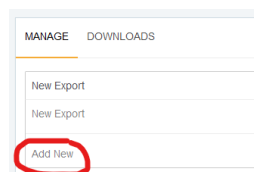
The image shows the 'Explore' interface in Learning Locker. On the left, there's a sidebar with a search bar and a filter section. The filter section has a 'Who' tab selected, and a search box containing '183B1CA9'. Below this, there are several expandable sections: 'Additional Data', 'Actor', 'Did', 'What', 'Where', 'Metadata', 'Result', 'When', and 'Store'. On the right, there's a table of events. The table has two columns: the first column contains event details (e.g., '183B1CA9 exited level') and the second column contains the time (e.g., '25 days ago').

Event	Time
> 183B1CA9 exited level	25 days ago
> 183B1CA9 launched level	25 days ago
> 183B1CA9 exited level	25 days ago
> 183B1CA9 completed level	25 days ago
> 183B1CA9 executed program	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago
> 183B1CA9 inserted block	25 days ago

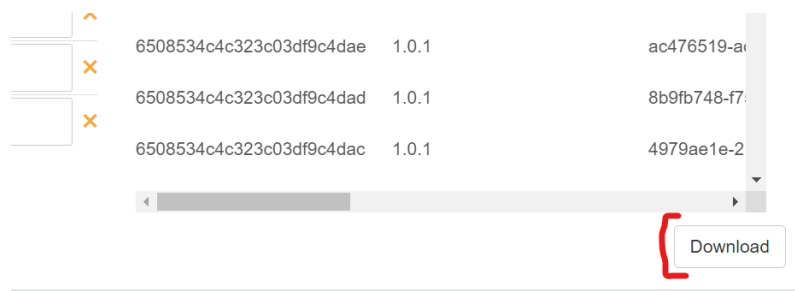
4. Cliquez sur le menu « Open export panel »



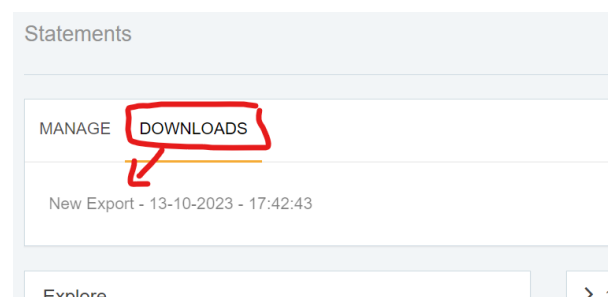
5. Ouvrez la liste déroulante et choisissez « Add New », nommez votre export pour pouvoir le retrouver ensuite.



6. Vous pouvez paramétrer votre export, pour l'instant laissez les champs par défaut et cliquez sur le bouton « Download »



7. Dans l'onglet « DOWNLOADS » téléchargez votre export



8. Vous récupérez un fichier CSV contenant les statements correspondant à votre filtre, ouvrez ce fichier avec Calc ou Excel et choisissez comme séparateur la virgule.
 - a. Note pour Excel, modifier l'extension du fichier en txt au lieu de csv avant de l'ouvrir pour forcer Excel à afficher la fenêtre d'import sans quoi il utilisera comme séparateur par défaut la tabulation
9. Explorez vos données et vérifiez si elles retracent bien votre session de jeu.

Communiquer avec LearningLocker en javascript

1. Editez le fichier « ExempleCommunicationLRS.html » livré avec ce tutoriel. Ce fichier contient une balise div avec comme id "result", et récupère en js cet élément au chargement de la page. Ce fichier charge également la bibliothèque "tincan.js" dont la documentation est

disponible à l'adresse suivante : <http://rusticsoftware.github.io/TinCanJS/doc/api/latest/>.

Votre objectif sera de construire une requête au LRS et récupérer les données de votre session de jeu pour les afficher dans la balise "result".

2. Connection avec le LRS : identifiez dans le fichier le commentaire

```
// Compléter ici...
```

Et remplacez-le par le code ci-dessous. Ce code ouvre une connexion avec le LRS, il faut à minima définir le endpoint (l'url où se trouve le LRS) ainsi que la clé et le mot de passe du client autorisé à faire la requête. Les valeurs données dans cet exemple sont celles du client autorisé à requêter le store « SPY_Prod » en lecture seule.

```
var myTinCan;
try{
  myTinCan = new TinCan();
  myTinCan.lrs = new TinCan.LRS(
    {
      endpoint: "https://lrsels.lip6.fr/data/xAPI",
      username: "6cf5a2a999bab59c72e325cf6d387df8e2ccaae3",
      password: "7033d7d702a42e9a062718eb6b63df6c0a71ba99"
    });
  htmlOutput.innerHTML = "Connection avec le LRS OK<br>";
}
catch (ex) {
  htmlOutput.innerHTML = "Failed to setup LRS object: " + ex;
}
```

3. Testez votre page avec un navigateur et vous devriez avoir le message « Connection avec le LRS OK » affiché à l'écran.
4. Construction et envoi d'une requête : après avoir établi la connexion avec le LRS nous allons utiliser la méthode « queryStatements » pour requêter le LRS, n'hésitez pas à consulter la doc de cette méthode¹ pour obtenir le détail des paramètres possibles. Dans cet exemple nous construisons une requête qui sélectionnera tous les statements d'un acteur particulier. Indiquez votre identifiant en lieu et place du "YOUR_SESSION_ID" pour récupérer les traces de votre session de jeu.

```
var myTinCan;
try{
  ...
  htmlOutput.innerHTML = "Connection avec le LRS OK<br>";
  // Construction d'une requête pour récupérer tous les statements d'un agent
  htmlOutput.innerHTML += "Envoi requête, en attente de réponse...<br>";
  myTinCan.lrs.queryStatements(
    {
      params: {
        agent: new TinCan.Agent({
          account: {
            "homePage": "https://www.lip6.fr/mocah/",
            "name": "YOUR_SESSION_ID"
          }
        }),
        limit: 100
      },
      callback: function (err, response) {
        if (err !== null) {
          console.log("Failed to query statements: " + err);
          return;
        }
        htmlOutput.innerHTML += "Nombre de statements reçus : " + response.statements.length + "<br>";
      }
    });
}
catch (ex) {
  htmlOutput.innerHTML = "Failed to setup LRS object: " + ex;
}
```

¹ Documentation de la méthode « queryStatements » :

http://rusticsoftware.github.io/TinCanJS/doc/api/latest/classes/TinCan.LRS.html#method_queryStatements

```
}
```

5. Testez votre page, vous devriez avoir comme information le nombre de statement de votre session de jeu.
6. Nous allons maintenant définir une fonction permettant de traiter la réponse envoyée par le LRS. Cette fonction parcourt simplement la liste des statements reçus pour les afficher sur la page web :

```
// Process LRS results
function processLrsResult(err, response, myTinCanLRS) {
  if (err !== null) {
    htmlOutput.innerHTML = "Failed to query statements: " + err;
    return;
  }

  htmlOutput.innerHTML += "Statements received: "+response.statements.length+"<br>";
  if (response.statements.length > 0){
    // enregistrement du statement
    for (let j = 0; j < response.statements.length; j++ )
      htmlOutput.innerHTML += response.statements[j]+"<br>";
  }
}
```

7. Appelez cette fonction dans la callback de la génération de la requête

```
myTinCan.lrs.queryStatements(
{
  params: {
    ...
  },
  callback: function (err, response) {
    processLrsResult(err, response, myTinCan.lrs);
  }
});
```

8. Testez votre page, vous devriez obtenir la liste des statements de votre session de jeu.
9. A noter que la réponse renvoyée par le serveur contiendra un nombre limité de statements, dans notre exemple 100 au maximum (cf paramètre "limit" lors de la construction de la requête). Il faut donc gérer le cas où votre requête générerait plus de statements que cette limite. L'API propose pour cela d'utiliser la méthode "moreStatements". Modifiez votre code comme suit :

```
// enregistrement du statement
for (let j = 0; j < response.statements.length; j++ )
  htmlOutput.innerHTML += response.statements[j]+"<br>";

// Vérifier si l'on doit récupérer plus de données
if (response.more !== null && response.more !== "") {
  htmlOutput.innerHTML += "Require to fetch more data<br>";
  myTinCanLRS.moreStatements({
    url: response.more,
    callback: function (err, response){
      processLrsResult(err, response, myTinCanLRS);
    }
  });
}
```

10. Testez votre page, pour vérifier si votre session de jeu contient plus de 100 statements
11. Nous allons afficher l'horodatage de chaque statement, pour cela modifiez le code comme suit :

```
if (response.statements.length > 0){
  // enregistrement du statement
  for (let j = 0; j < response.statements.length; j++ )
    htmlOutput.innerHTML += response.statements[j]+"
"+response.statements[j].timestamp+"<br>";
}
```

12. Testez votre page et observez les timestamps, il est possible que l'ordre des statements dans la réponse du serveur ne respecte pas l'ordre temporel de création des statements. Nous allons trier les statements par ordre chronologique :

a. Créez un tableau pour stocker les statements

```
var htmlOutput;
var sessionData = new Array();
```

b. Remplacez l'affichage du statement par son ajout dans le tableau

```
// enregistrement du statement
for (let j = 0; j < response.statements.length; j++) {
    sessionData.push(response.statements[j]);
}
```

c. Triez et affichez les statements une fois qu'ils ont tous été récupérés

```
// Vérifier si on doit récupérer plus de données
if (response.more !== null && response.more !== "") {
    ...
}
else {
    // trier les statement du plus ancien au plus récent
    sessionData.sort(function(from, to){
        date1 = new Date(from.timestamp);
        date2 = new Date(to.timestamp);
        return date1 - date2;
    });
    // Affichage des statements triés
    for (let j = 0; j < sessionData.length; j++) {
        htmlOutput.innerHTML += sessionData[j] + " " + sessionData[j].timestamp + "<br>";
        // Affichage du statement dans la console
        console.log(sessionData[j]);
    }
}
```

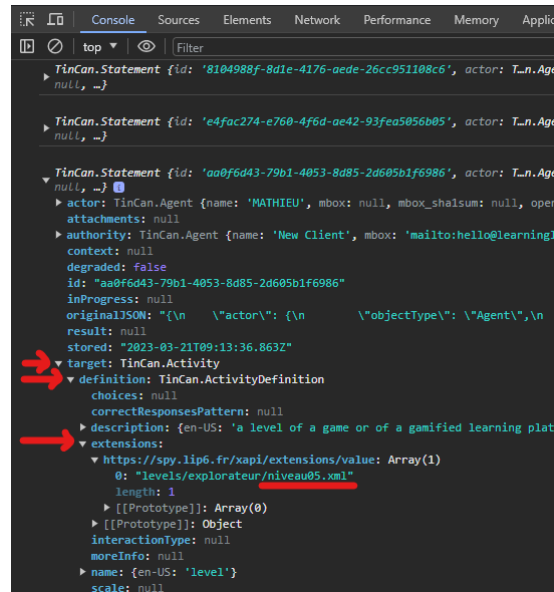
13. Testez votre page, les statements devraient être maintenant affichés par ordre chronologique.

14. La requête que nous avons construite dans ce tutoriel filtre les statements d'un utilisateur particulier. Vous pouvez affiner vos requêtes en filtrant un verbe particulier, un objet particulier... Par exemple en ajoutant les paramètres suivants lors de la construction de la requête, vous ne récupérerez que les statements de votre session avec le verbe « launched » (voir le vocabulaire utilisé par SPY² pour connaître les id possibles). C'est un bon moyen de reconstruire le parcours d'un utilisateur :

```
myTinCan.lrs.queryStatements(
{
    params: {
        agent: new TinCan.Agent({
            account: {
                "homePage": "https://www.lip6.fr/mocah/",
                "name": "YOUR_SESSION_ID"
            }
        }),
        verb: new TinCan.Verb({
            id: "http://adlnet.gov/expapi/verbs/launched"
        }),
        limit: 100
    },
    callback: function (err, response) {
        processLrsResult(err, response, myTinCan.lrs);
    }
});
```

² Vocabulaire utilisé par SPY : <https://spy.lip6.fr/xapi/vocabulary.html>

15. Observez la console de votre navigateur et explorez la structure des statements reçus pour savoir quels niveaux ont été lancés et dans quel ordre.



```

TinCan.Statement {id: '8104988f-8d1e-4176-aede-26cc951108c6', actor: T.n.Age
null, -}

TinCan.Statement {id: 'e4fac274-e760-4f6d-ae42-93fea5056b05', actor: T.n.Age
null, -}

TinCan.Statement {id: 'aa0f6d43-79b1-4053-8d85-2d605b1f6986', actor: T.n.Age
null, -}
  ▶ actor: TinCan.Agent {name: 'MATHIEU', mbox: null, mbox_sha1sum: null, oper
  ▶ attachments: null
  ▶ authority: TinCan.Agent {name: 'New Client', mbox: 'mailto:hello@learning1
  ▶ context: null
  ▶ degraded: false
  ▶ id: "aa0f6d43-79b1-4053-8d85-2d605b1f6986"
  ▶ inProgress: null
  ▶ originalJSON: "{\n  \"actor\": {\n    \"objectType\": \"Agent\", \n
  ▶ result: null
  ▶ stored: "2023-03-21T09:13:36.863Z"
  ▶ target: TinCan.Activity
  ▶ definition: TinCan.ActivityDefinition
    ▶ choices: null
    ▶ correctResponsesPattern: null
    ▶ description: {en-US: 'a level of a game or of a gamified learning plat
    ▶ extensions:
      ▶ https://spy.lip6.fr/xapi/extensions/value: Array(1)
        0: "levels/explorateur/niveau05.xml"
        length: 1
      ▶ [[Prototype]]: Array(0)
      ▶ [[Prototype]]: Object
      ▶ interactionType: null
      ▶ moreInfo: null
      ▶ name: {en-US: 'level'}
      ▶ scale: null

```

16. Créez vos propres requêtes et explorez les données reçues/
- Par exemple explorer les traces d'un autre étudiant
 - Récupérez les codes de session de tous les joueurs ayant joué au jeu depuis de début de cette séance
 - Parmi les 100 dernières missions lancées par tous les utilisateurs identifiez la plus jouée
 - ...